Software Engineering Lab

UE24CS341A

Lab 4: Vibe Coding — Chat History

Rhythm Tap (Pygame) | Implementation, debugging, testing and GitHub workflow

Name: HARSHITHA.B.T	SRN: PES1UG24AM118	Section: AIML - "B"

You

We are working on Software Engineering Lab 4 (VibeCoding) for the Rhythm Tap Pygame project. The assigned starter repository is https://github.com/SETAPESU26/51_rhythm-tap, and my personal repository is https://github.com/harshithabt/51_rhythm-tap. okay now lab04 i am PES1UG24AM118

Assistant

Recommended using a separate repository for Lab 4, cloning the starter project locally, reading the README requirements, running the game before editing, recording a 10-second BEFORE video, and implementing each requested feature separately with testing and commits.

You

I installed Git and cloned the project. I am working in Windows Command Prompt and the game launches.

Assistant

Confirmed that the project should be run locally because Pygame needs a graphical window. Suggested testing the starter game and capturing the BEFORE recording before making code changes.

Assistant

Your starter project is now in your own GitHub repository, so we can move straight to the actual Lab 4 work.

We'll keep this fast and follow the assignment requirements: baseline video → four features → testing → final video → chat history and submission.

Step 1 — Run the original game first

Keep your CMD open in the 51_rhythm-tap folder.

Run these commands one at a time:

python --version

python -m pip install -r requirements.txt

python main.py

If python isn't recognized, try py instead.

Before changing any code, record 10 seconds of the original gameplay as required by your lab handout. If the game crashes or something is broken, capture that behavior if possible.

Step 2 — Show me the starter code

After recording the video, run these commands in CMD:

type main.py

type game\game_engine.py

type game\beat.py

Copy the output here (if long, send one file at a time). I'll use your actual starter code to guide the changes instead of guessing and risking breaking the game.

Our four-task checklist

- Task 1 — Sound effects for PERFECT, GREAT, and OK hits
- Task 2 — Hold notes requiring one second
- Task 3 — BPM-synchronized note spawning

- Task 4 — Grade summary showing counts and accuracy
- Separate Git commit for each task
- 10-second before and after videos
- Exported chat history and submission in your existing Lab04 folder

First, tell me whether the game launches successfully. If it does, record the before video and send me main.py next. We'll work through the features directly.

You

Assistant

Okay, this is an easy fix! Your Git setup is fine. The error simply means Pygame hasn't been installed yet, so the game can't start.

Let's fix it first.

Step 1 — Install the game dependencies

In your current CMD window, you're already in the correct folder:

C:\Users\Harshitha.B.T\51_rhythm-tap

Run this command:

python -m pip install -r requirements.txt

Wait for it to finish. Your requirements.txt should specify the dependencies needed by the game.

Step 2 — Launch the game again

python main.py

If the game window opens, great! Play for a few seconds and record your 10-second BEFORE video before changing any code.

If you still see No module named 'pygame', run:

python -m pip install pygame

Then retry:

python main.py

Step 3 — Tell me what happens

- Game opens: Record the baseline video, then we'll inspect the code and implement the four tasks.
- Another error appears: Send me a screenshot of the error, and we'll fix it directly.

We're close to the actual coding now!

You

I finished playing the game. What do you mean by watching for a long note?

Assistant

Explained that a hold note is visually taller than a normal note. When it reaches the hit line, press and keep holding the matching lane key (D, F, J or K) continuously for one second.

You

I kept pressing the hold note and it said HOLD!! but didn't consider a point.

Assistant

Explained that displaying HOLD! only indicates that the hold started; the hold completion logic must award points after the full duration. The active hold must not be allowed to fall through the hit line and be marked as missed while the player is holding it.

You

Shared the current game_engine.py and asked whether it was correct.

Assistant

Reviewed the code structure and identified that the hold-note logic and update order needed to be consistent. Recommended compiling the files and testing the hold mechanic before proceeding.

You

Shared a screenshot and said the same problem remained: holding the key still did not score or play a sound.

Assistant

Asked to inspect game/beat.py as well, since the note class defines the hold duration and the visual height. The two files need to agree on the hold-note behavior.

You

Shared game/beat.py. The Note class had is_hold, hold_duration = 1.0, and get_rect() made hold notes three times as tall as normal notes.

Assistant

Provided a complete game_engine.py replacement with sound generation, hold tracking, key-down/key-up handling, a hold update method, and a run() method. The intended behavior was to freeze an active hold note while the key remains down, award points after one second, and play the PERFECT sound.

You

Said the hold note seemed to be working but you were not able to see the result clearly, then move ahead.

Assistant

Moved on to BPM-synchronized spawning and the game-over grade summary. At 120 BPM, one beat is 60,000 / 120 = 500 milliseconds. Spawn timing should use elapsed time rather than relying only on a frame counter.

You

Asked for the final .py file.

Assistant

Provided a complete file intended to include sound effects for PERFECT/GREAT/OK, one-second hold notes, 120 BPM spawning, and a game-over summary showing PERFECT, GREAT, OK, MISS and accuracy.

You

Shared a terminal screenshot showing an AttributeError that GameEngine had no attribute run, Git author identity unknown, and a rejected push.

Assistant

Explained that main.py calls GameEngine.run(), so the method needed to be present and correctly indented inside the GameEngine class. Git also needed a configured author name/email, and the rejected push required synchronizing the local branch with origin/main.

You

Shared screenshots showing the game running and the grade summary.

Assistant

Confirmed that the game-over screen and BPM display were visible. Recommended testing successful hits, sound playback, hold-note scoring, and the final summary before completing Git setup.

You

Asked whether to repeat git add and git commit because those commands had already been attempted.

Assistant

Recommended checking git status and git log first, rather than creating duplicate commits blindly.

You

Shared a gameplay screenshot showing Score 1800, PERFECT 5, GREAT 0, OK 3, MISS 15, Accuracy 34.8%, Max Combo 3x, and BPM 120.

Assistant

Confirmed that the game was scoring successful hits and that the sound effects worked. Accuracy matched 8 successful hits out of 23 judgements: 8 / 23 × 100 ≈ 34.8%.

You

Shared Git status showing modified game_engine.py and beat.py and a new .gitignore.

Assistant

Recommended staging the intended source files, adding __pycache__/ and *.py[cod] to .gitignore, committing the feature changes, and then pulling with rebase before pushing.

You

Shared another terminal screenshot where the commit failed because Git identity was not configured and the pull failed because there were uncommitted changes.

Assistant

Recommended configuring git config --global user.name and user.email with the user's actual GitHub details, committing after configuration, then running git pull --rebase origin main and git push origin main. Advised stopping if a merge conflict appeared.

You

Shared terminal output confirming the commit and push succeeded. The final feature commit shown was c9823f0, with the local main branch synchronized to origin/main and a clean working tree.

Assistant

Confirmed that the code was pushed successfully. Listed the remaining submission items: BEFORE video, AFTER video, chat history PDF/document, README and any other Lab-4 deliverables required by the handout.

You

Asked how to show the grade summary in a 10-second AFTER video.

Assistant

Recommended recording gameplay and a successful hit, then allowing the game to reach GAME OVER so the grade summary appears. If necessary, record a longer clip and trim it to the required duration, ensuring the final summary remains readable.

You

Confirmed that the AFTER video was done and asked about PDF containing the main things from the chat.

Assistant

Created a concise summary PDF, but that was not the required format.

You

Clarified that the instructor wants the chat history itself as a PDF.

Assistant

Created a transcript-style PDF with the main conversation exchanges, and explained that an exact verbatim export is best obtained through ChatGPT's own export/print feature.

Reviewed the examples. They use a dialogue format with user prompts, assistant responses, relevant code/explanations and placeholders for screenshots or uploaded files, rather than a short project summary.

Repository

https://github.com/harshithabt/51_rhythm-tap

Key final commit shown in terminal: feat: add BPM spawning and grade summary.

Important note: This is a detailed transcript-style reconstruction of the key exchanges available in this conversation. It is not a verbatim ChatGPT export and does not reproduce every long code paste or screenshot. For a literal full transcript, use ChatGPT's chat export/print feature.